Instruction for project 2

Basics of Computer Programming 2025/26, Data Engineering

author: Robert Ostrowski[1]
date: 22/11/2025

Beat This Project

The goal

The goal of the project is to implement a single-player scrolling beat'em up game (https://en.wikipedia.org/wiki/Beat_%27em_up). Examples of the genre can be found on the aforementioned page. The character traverses the stage and beats the enemies who wish to hinder the completion of the project and passing the course. Selected functionalities/game elements that have to be implemented are listed below.

It is also possible to change the theme of the game to be less violent (attacks can instead represent actions that match the description of the mechanics of the game). The following series of articles on the adjacent genre of fighting games contains additional information you might find helpful.

Programming Environment

The instruction comes with a template program that implements:

- time increment calculation, which allows you to track its passage;
- displaying graphic files in the BMP format on the screen;
- drawing pixels, lines and rectangles;
- text display.

The program utilizes the SDL2 library (2.0.3) – http://www.libsdl.org/. It is included in the helper program, and there is no need to download its sources.

Compilation under Linux is performed using the following command (in a 32-bit system):

```
g++ -O2 -I./sdl/include -L. -o main main.cpp -lm -lSDL2 -lpthread -ldl -lrt
```

or (in a 64-bit system):

```
g++ -O2 -I./sdl/include -L. -o main main.cpp -lm -lSDL2-64 -lpthread -ldl -lrt
```

[1] Note: In case of any ambiguity in the instruction, please contact the author under the address robert.ostrowski@pg.edu.pl; further contact information can be found in the "Basics of Computer Programming 2025/2026" course on enauczanie.

In order to successfully compile the helper program, the main.cpp file should be in the directory with

- bitmaps with required drawings (cs8x8.bmp, eti.bmp);
- libSDL2.a file (libSDL2-64.a for 64-bit compilation);
- sdl directory attached to the program.

The program includes scripts that can be used for compilation (comp in a 32-bit environment and comp64 in a 64-bit environment).

The presentation of the program (required if you want to get any points) will take place in an environment chosen by you from the following two options:

- Linux: you should check, before coming to the examination, whether the program compiles and runs correctly under the distribution available in the laboratory;
- Windows: in the MS Visual C++ environment, in a version compatible with the one available in the laboratory.

Showing that your program runs (during the presentation) is a necessary condition for obtaining any points.

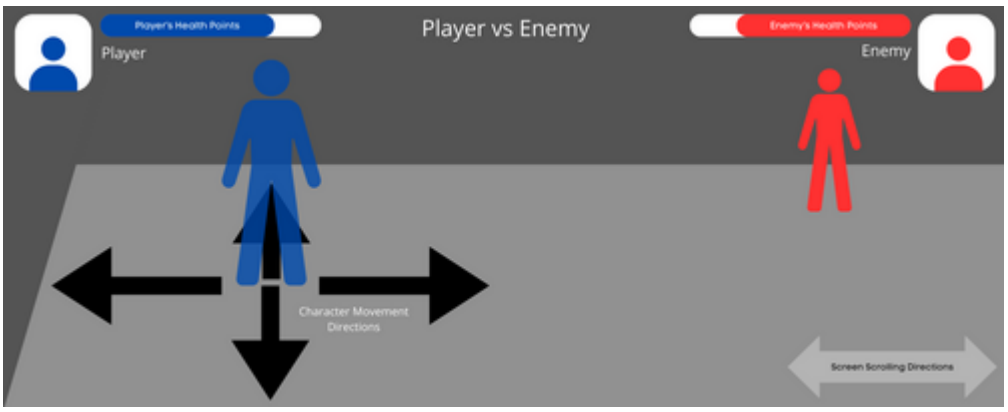
Mandatory Requirements (5 points)

All items listed here must be implemented. The version of basic requirements is very simplified; in particular, the control can be used to test further requirements. The absence of any of the following elements results in no points for this project.

1. Preparing the graphic design of the game: outline of the scene, preparing a place to display additional information (e.g. time that has passed since the beginning of the stage, actions undertaken by the player). Control key implementation:
   a. ***Esc***: exit from the program - the program ends immediately;
   b. ***'n'***: new game.

2. Implementation of one stage of the game. The stage should have a floor of a limited size and a background. The scene size should not be limited to the window size, i.e. the camera should follow the character (leave enough grace space to keep the field of view reasonable).

3. Implementation of the mechanics of moving the character using arrow or WSAD keys. The position of the character should be limited by the underlying resolution of the game, not the position of the blocks from which the scene is built. **Note:** Barring implementation of extra points J**,** it is sufficient to track the position of characters in 2 dimensions, height of characters is not relevant mechanically.



4. The elapsed time should be measured properly and displayed along with the information about implemented requirements (mandatory and optional).

Optional Requirements (10 points)

A. **(1 pt)** Implementation of jumping and attacks:
   a. After pressing an appropriate button, the character should perform an action, at least:
      i. a light action or attack (*action X*) - faster
      ii. a heavy action or attack (*action Y*) - slower
   b. Character movement and control parameters should be easy to change to achieve smooth control.
   c. If animations (point D) are not implemented, replacing the sprite or changing a color for a fixed number of frames upon actions A and B is sufficient.

B. **(1 pt)** Combo input
   a. Implement an input buffer for commands (an example of the idea can be found here)
   b. At least 3 additional actions should be accessible by combining the basic input in the following fashion:
      i. pressing either the action X or Y button three times in a row
      ii. a specific sequence of X and Y action buttons
      iii. a dash - quick change of position in a given direction based on pressing movement buttons alone.
   c. The game does not have to provide the ability for the user to add combos; however, the code is expected to be written in a way that allows the programmer to easily add further combinations.
   d. Pressing a developer mode key should display the buffer, its contents and the name of the currently performed action.



C. **(1 pt)** Hitboxes:
   a. Implement a game object that can be interacted with (replaced by enemies in point E).
   b. Action X should have a different hitbox and speed than action Y. All combo actions from point B should result in a move with distinct

characteristics.

c. A hit should award some number of points, with each consecutive hit (without being hit) in a time window increasing the awarded points multiplier.

D. **(2 pts)** Animations (requires points A and C described above):
   a. Animations of walking, being hit and actions of the character and enemies. You can keep the number of different frames to the minimum while maintaining readability
   b. The current points multiplier should appear and increment on the screen in a bold and flashy way.



   c. **Note:** animation speed should not depend on the speed of the computer (assuming that it meets the minimum requirements of the game). Naive solution from the point A.c is insufficient!

E. **(1 pt)** Enemies
   a. A limited number of enemies of at least 2 types appear in each stage.
   b. One enemy type follows a simple behaviour pattern of closing up to and attempting to hit the player character. The second type should keep their distance and line up with the player and perform a charge.
   c. Landing a hit on the enemy should result in a brief interruption of their action (stun).

F. **(2 pts)** Encoding the stage in a file:
   a. The file should contain the following elements of the stage configuration:
      i. width and obstacles: the stage should have a varying width at different points and contain additional obstacles to the player's movement.
      ii. placement and number of enemies: should be stage dependent (if point E is not implemented, instead place immovable dummy targets.
   b. Each stage should have a varying width at different points and contain additional obstacles to the player's movement.
   c. Placement and number of enemies should be stage dependent (if point E is not implemented, instead place immovable dummy targets.
   d. After the enemies are defeated, the program should detect entering the exit area, ending the stage and signal this event.
   e. **Note**: the program should not impose limits on the maximum number of objects of various types contained in the file. This means that the program analyzes the contents of the file and then allocates enough memory to store data about all the objects in the file. The format for encoding this information in the file should be selected by you, which means that to make reading the stage description from the file easier,

you can choose a format in which, there is information at the beginning of the file (preamble) about the number of individual objects, and then there is the description of objects. This way, you can load the preamble, allocate memory based on the data there, and then load the data. (With this solution, data validation should be added - the program should recognize any inconsistency between the preamble and the content.)

G. **(1 pt)** Menu and interface:
   a. After starting the game, a menu should appear allowing the player to select all options: exit, check results (point H), and select a stage (point F). Selecting an option that is not implemented should display a message stating that it is unavailable.
   b. The text entered by the player (e.g. during writing a nickname) should always be visible, and the ***backspace*** key should allow removing letters.
   c. Display the hit points bar of the player character on the screen in graphical form.
   d. Losing all hit points should prompt the player to continue or go back to the menu and show (if point F is implemented) the number of points the player has gained.
   e. After reaching the end of a stage, proceed to the next one (prepare a simple way to demonstrate this functionality in advance).

H. **(1 pt)** Saving best scores:
   a. Once a player finishes playing, they should be able to enter a nickname and score into the file. The number of results saved in the file should not be limited.
   b. A player can view a sorted list of results from the menu or after a specific button is pressed.
   c. The number of results on the screen, as opposed to those contained in the file, should be limited.
   d. If not all results fit on the screen, the program should allow the player to reach them, e.g. by switching between pages or scrolling through the list.
   e. If point C is not implemented, time can be recorded instead of results.

Extra Requirements (3 points)

Note: the following requirements are only assessed when all other points have been implemented.

I. **(1,5 pt)** Jump - third dimension
   a. Add a jump action.
   b. It should be possible to jump over the enemies.
   c. Pressing either the action X or Y button while in the air should result in an execution of a specific move.

J. **(1,5 pt)** Encoding the game controls and actions, and combos in a file:

a. You should design your own (editable, e.g. in a text editor) control schema file format. This format should be familiar to you so that you are able to explain and make the indicated edits during the presentation.
b. The file should specify: how to perform a given move, hitboxes, and damage values. Furthermore, it should be possible to grant access to them to specific types of enemies.

Final Remarks

The program configuration should enable easy change (i.e. modifying a constant) of all parameters, not only those clearly indicated in the above description.

The program can be written in an object-oriented way, but it is strictly forbidden to use the following C++ standard library components: string type, cin, cout, vector and other stl containers (Note: the string type from the C++ library should not be confused with the string.h library from C – you can use functions located in string.h)

File handling should be implemented using the C standard library (f???? family of functions - e.g. fopen, fread, fclose, etc.)

The speed of the program should be independent of the computer on which the program is running (provided that the minimal requirements are met).

The quality of the code will be assessed according to the general requirements on the site of the course.